

Dart Assignment

Module 1: Introduction to Mobile Development and Flutter

1. Benefits of using Flutter over other cross-platform frameworks

Flutter offers several advantages compared to frameworks like React Native or Xamarin:

Single Codebase

Write one code and run on Android, iOS, Web, and Desktop.

Fast Performance

Uses native compilation (no bridge like React Native), so apps are smoother.

Hot Reload

Instantly see code changes without restarting the app.

Rich UI (Widgets)

Comes with customizable widgets for attractive UI design.

Open Source & Backed by Google

Strong community and regular updates.

Consistent UI Across Platforms

Same design on Android and iOS (no dependency on native components).

2. Role of Dart in Flutter & its advantages

Dart is the programming language used in Flutter.

Role:

Used to write Flutter apps

Builds UI using widgets

Handles logic, state, and backend interaction

Advantages:

Fast Execution → Compiled to native code

Hot Reload Support → Improves development speed

Easy Syntax → Similar to Java/C#

Strong Typing → Reduces errors

Asynchronous Programming → Better for APIs and UI responsiveness

3. Steps to set up Flutter development environment

Step 1: Install Flutter SDK

Download from: <https://flutter.dev>

Extract and set PATH

Step 2: Install required tools

Install Android Studio or VS Code

Install Flutter & Dart plugins

Step 3: Setup Android Emulator

Open Android Studio → AVD Manager → Create Virtual Device

Step 4: Verify installation

flutter doctor

Step 5: Fix any issues shown

Install missing dependencies

4. Basic Flutter App Structure

main.dart

Entry point of Flutter app

Contains main code

► main() function

```
void main() {  
  runApp(MyApp());  
}
```

Starts the app

Calls root widget

Widget Tree

Flutter UI is made of widgets

Everything = widget (Text, Button, Layout)

Example:

MaterialApp

└─ Scaffold

└─ Center

└─ Text

Module 2: Dart Programming Essentials

1. Fundamental Data Types in Dart and Their Uses

Dart is a statically typed language, which means every variable has a type.

int (Integer)

Used for whole numbers.

```
int age = 21;
```

```
int marks = 95;
```

Use:

Age

Count

IDs

Quantity

double

Used for decimal numbers.

```
double price = 99.99;
```

```
double pi = 3.14;
```

Use:

Price

Percentage

Measurement

String

Used for text.

```
String name = "Nandan";
```

```
String city = "Ahmedabad";
```

Use:

Names

Messages

Email

Labels

bool

Used for true/false values.

```
bool isLogin = true;
```

```
bool isPassed = false;
```

Use:

Login status

Conditions

Verification

List

Used to store multiple values in order.

```
List<String> fruits = ["Apple", "Banana", "Mango"];
```

Use:

Product lists

Student names

Tasks

Map

Stores data in key-value pairs.

```
Map<String, String> student = {  
  "name": "Nandan",  
  "city": "Rajkot"  
};
```

Use:

JSON data

User profiles

Database records

dynamic

Can store any type.

```
dynamic value = 10;
```

```
value = "Hello";
```

Use:

Flexible input

Unknown type data

2. Control Structures in Dart

Control structures manage decision-making and loops.

if Statement

Executes code if condition is true.

```
int age = 18;
```

```
if (age >= 18) {  
  print("Eligible to vote");  
}
```

if-else Statement

```
int marks = 40;
```

```
if (marks >= 35) {  
  print("Pass");  
} else {  
  print("Fail");  
}
```

for Loop

Repeats code fixed number of times.

```
for (int i = 1; i <= 5; i++) {  
  print(i);  
}
```

Output:

1 2 3 4 5

while Loop

Runs while condition is true.

```
int i = 1;
```

```
while (i <= 3) {  
  print(i);  
  i++;  
}
```

switch Statement

Used for multiple conditions.

```
String day = "Monday";
```

```
switch (day) {  
  case "Monday":  
    print("Start of week");  
    break;  
  case "Sunday":  
    print("Holiday");  
    break;  
  default:  
    print("Normal day");  
}
```

3. Object-Oriented Programming Concepts in Dart

Dart fully supports OOP.

Class

Blueprint for objects.

```
class Student {  
  String name = "Nandan";  
  
  void display() {  
    print(name);  
  }  
}
```

Object

```
void main() {  
  Student s = Student();  
  s.display();  
}
```

Inheritance

One class inherits another.

```
class Animal {  
  void sound() {  
    print("Animal sound");  
  }  
}
```

```
class Dog extends Animal {  
  void bark() {  
    print("Bark");  
  }  
}
```

Polymorphism

Same method, different behavior.

```
class Animal {  
  void sound() {  
    print("Animal sound");  
  }  
}
```

```
class Dog extends Animal {  
  @override  
  void sound() {
```

```

    print("Bark");
  }
}

```

Interface

Dart uses classes as interfaces.

```

class Printer {
  void printData() {}
}

```

```

class PDFPrinter implements Printer {
  @override
  void printData() {
    print("Printing PDF");
  }
}

```

4. Asynchronous Programming in Dart

Used for tasks like:

API calls

File download

Database fetch

Future

Represents a value available later.

```

Future<String> fetchData() {
  return Future.delayed(
    Duration(seconds: 2),
    () => "Data Loaded",
  );
}

```

async

Marks function asynchronous.

await

Waits for Future result.

```

void main() async {
  String data = await fetchData();
  print(data);
}

```

Stream

Multiple asynchronous values over time.

```

Stream<int> countStream() async* {
  for (int i = 1; i <= 3; i++) {
    yield i;
  }
}

```

Listen:

```

void main() async {
  await for (int value in countStream()) {
    print(value);
  }
}

```

Module 1 Practical:

<https://github.com/popatnandan2136/assignmentdart/tree/4d8c63e1292a9c2a66b07de72929b18921052806/lib/module1>

Module 2 Practical:

<https://github.com/popatnandan2136/assignmentdart/tree/4d8c63e1292a9c2a66b07de72929b18921052806/lib>